

Relatório — Ingestão Automatizada na Nuvem (Telemetria Aeroespacial)

Matéria: Plataformas, Serviços Cognitivos & Cloud Computing

Tema: API na nuvem com RAG

Alunos:

- Guilherme Orugian — RM 572882
- Rodrigo Bettio — RM 573725
- Rafael Jun — RM 569708

Data: 09/06/2026

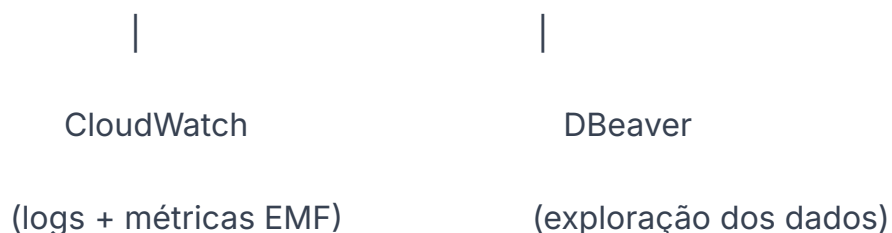
Repositório: <https://github.com/orugian/gs-cloud>

1. Objetivo

Planejar e implementar uma solução **serverless e automatizada** que recebe arquivos de **telemetria aeroespacial** em um **bucket S3** e, sem nenhuma intervenção manual, processa e grava os dados em uma **base relacional RDS MySQL**. A automação é evidenciada pelo **Amazon CloudWatch** (logs e métricas) e os dados são explorados via DBeaver conectado ao endpoint do RDS. O modelo de dados já é construído de forma a suportar uma camada de **RAG** em etapa futura.

2. Arquitetura

upload .csv —► S3 (bucket) —► (evento ObjectCreated) —► Lambda (Python 3.12) —► RDS MySQL 8



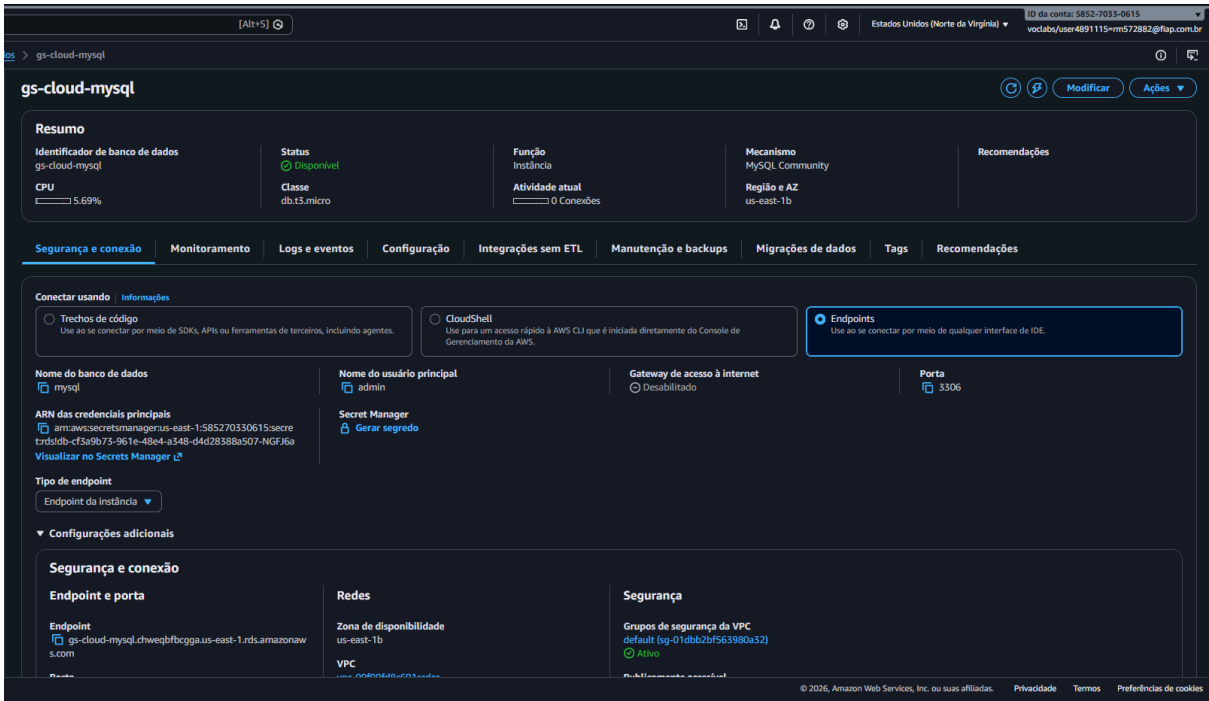
O evento `ObjectCreated` do S3 invoca a Lambda automaticamente. A função valida cada linha, resolve a missão via upsert, insere a telemetria em lote com `INSERT IGNORE` (idempotente) e registra a execução em `ingestao_log`. As métricas são emitidas em **Embedded Metric Format (EMF)** — a Lambda imprime JSON estruturado no stdout e o CloudWatch extrai as métricas automaticamente, sem chamada de API.

3. Serviços AWS utilizados

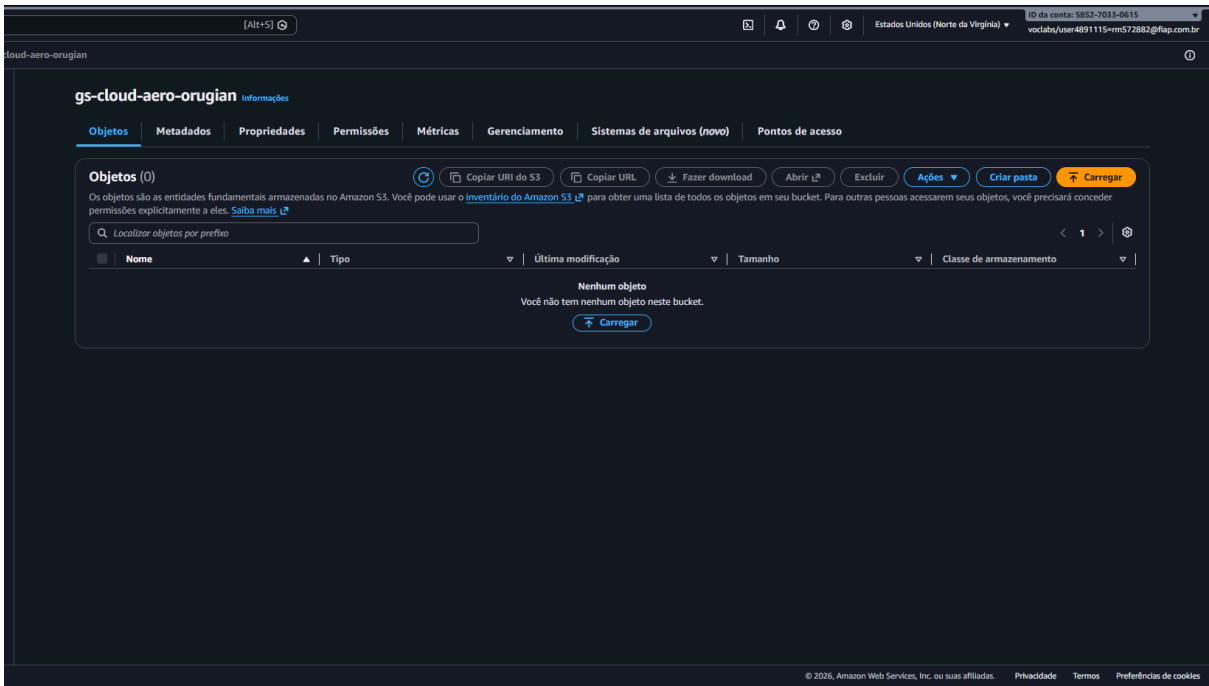
Serviço	Papel
Amazon S3	Zona de aterrissagem (landing zone) dos arquivos <code>.csv</code> .
AWS Lambda (Python 3.12)	Ingestão serverless: valida, processa e grava no RDS.
Amazon RDS (MySQL 8)	Base relacional de destino com 4 tabelas normalizadas.
Amazon CloudWatch	Logs estruturados e métricas customizadas (EMF).
IAM (LabRole)	Permissões de execução da função Lambda e acesso a recursos.

4. Execução e evidências

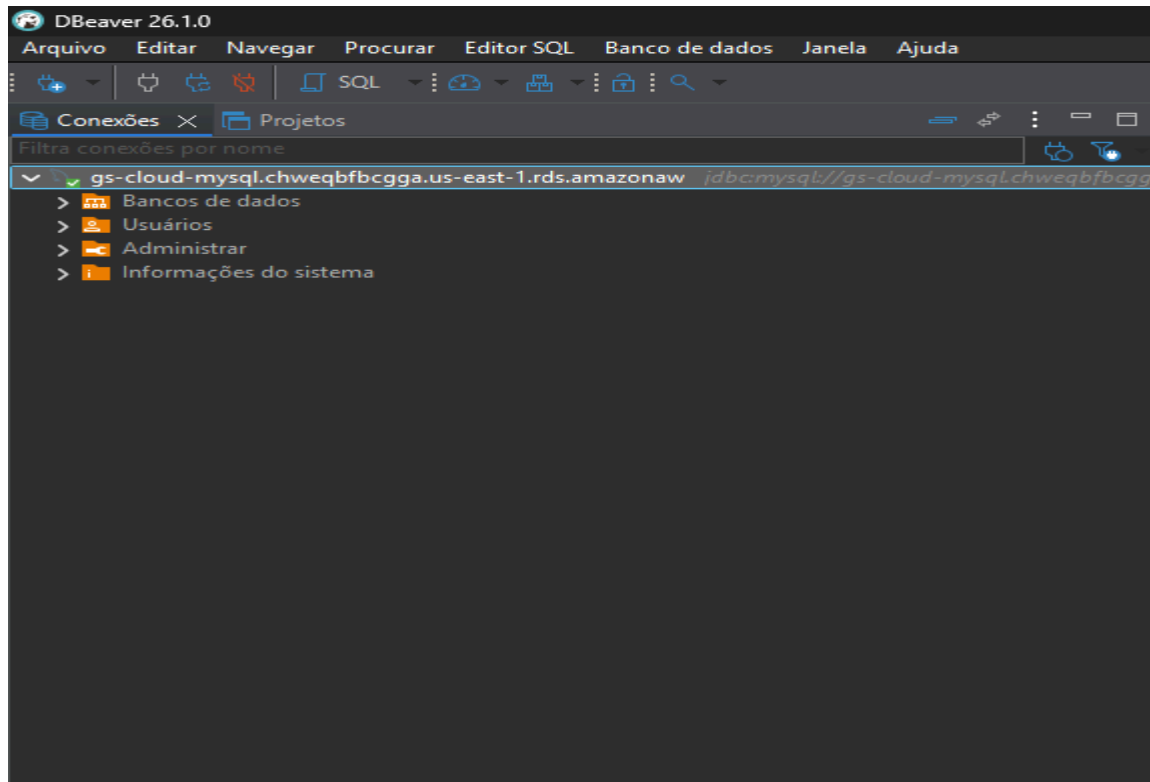
4.1 RDS provisionado

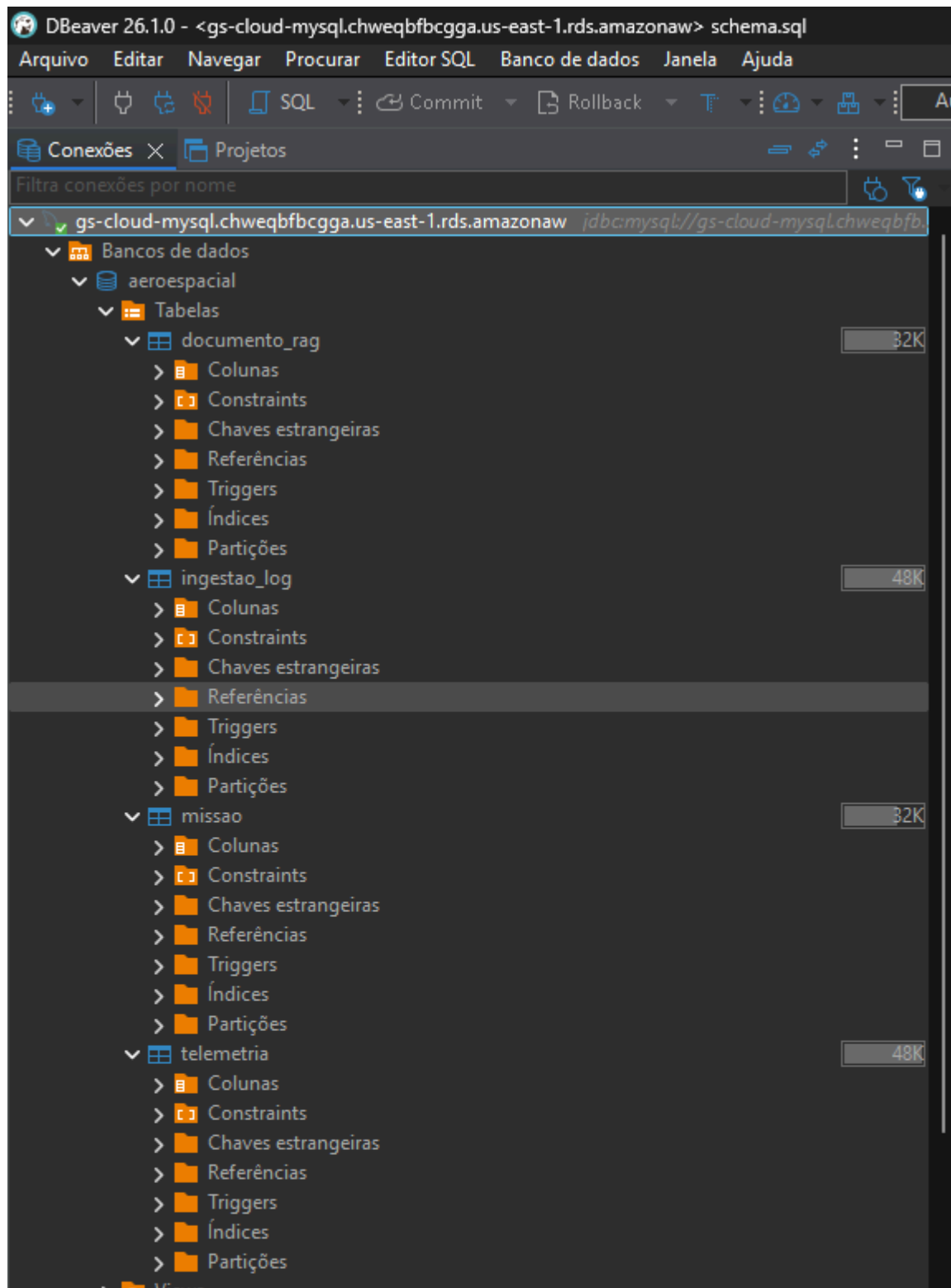


4.2 Bucket S3



4.3 Conexão ao banco e criação do schema





4.4 Função Lambda

Interface do AWS Lambda console mostrando a configuração da função **gs-cloud-ingestao**.

Configuração

- Configuração geral
- Gatilhos
- Permissões
- Destinos
- URL da função
- Variáveis de ambiente**
- Tags
- VPK
- Bancos de dados do RDS
- Ferramentas de monitoramento e operações
- Detecção de recurso e simultaneidade
- Invocação assíncrona

Variáveis de ambiente (5)

As variáveis de ambiente a seguir são criptografadas em repouso com a chave de serviço padrão do Lambda.

Chave	Valor
CW_NAMESPACE	GScloud/ingestao
DB_HOST	gs-cloud-mysql.chwedfbcgga-us-east-1.rds.amazonaws.com
DB_NAME	aerospacial
DB_PASSWORD	postgres
DB_PORT	3306
DB_USER	admin

Interface do AWS Lambda console mostrando o código da função **gs-cloud-ingestao**.

Propriedades do código

- Tamanho do pacote: 129,6 KB
- Hash SHA256: j56NTSjE5BF8a2FF36OBNsqndmY3mQ7YG05ydaAV1Y=
- Última modificação: em 18 segundos

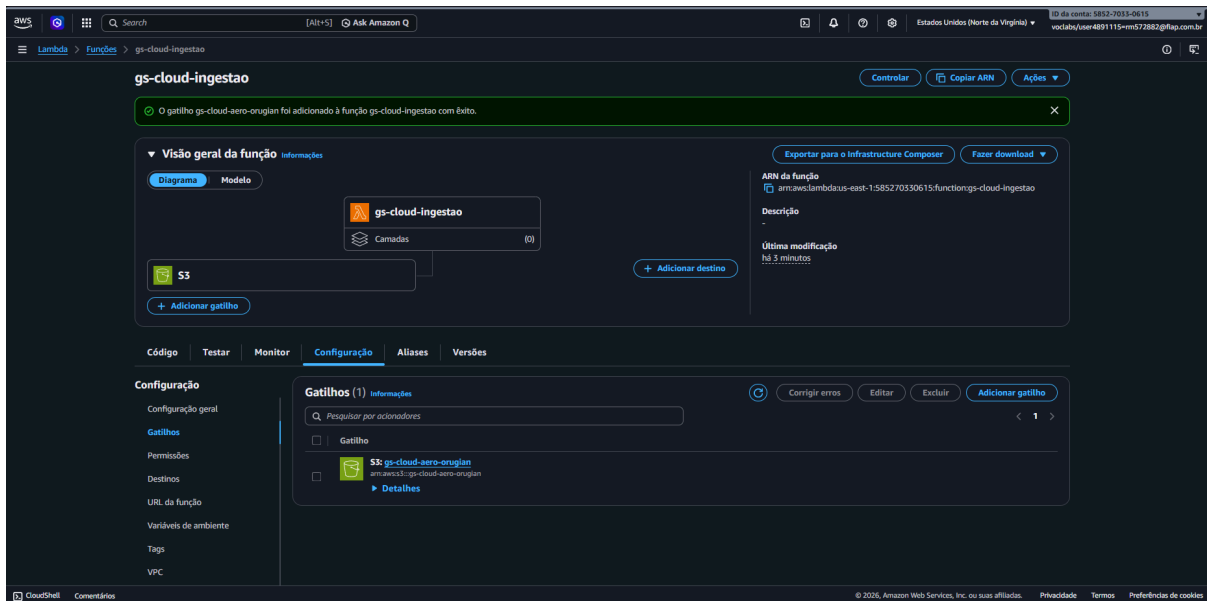
Configurações do runtime

- Runtime: Python 3.14
- Manipulador: handler.lambda_handler
- Arquitetura: x86_64

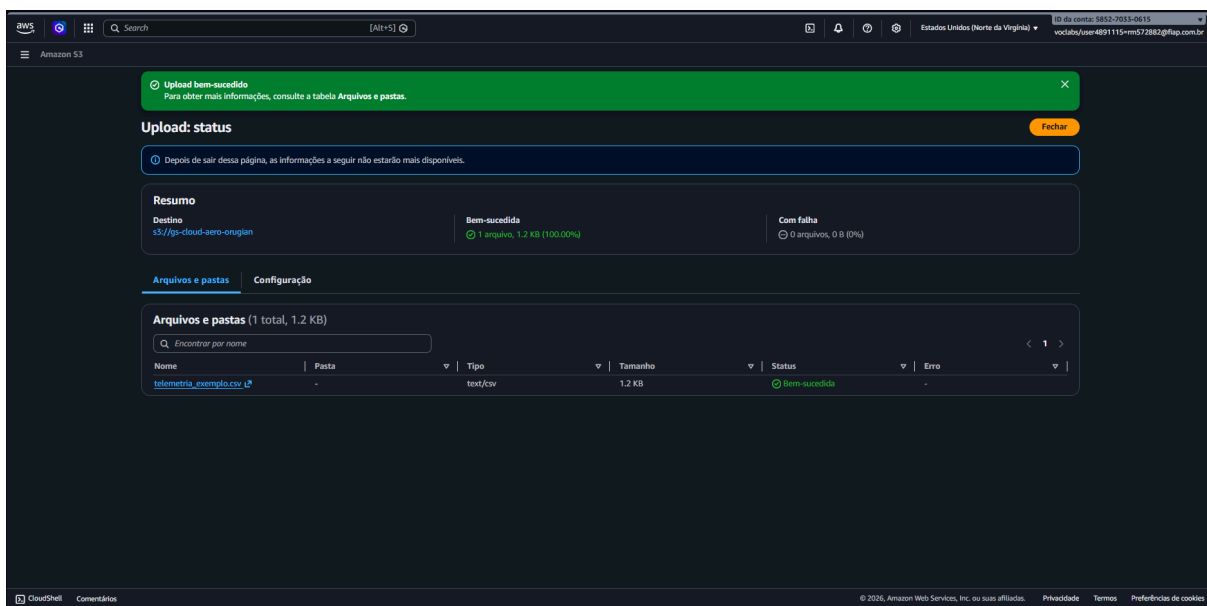
Configuração de gerenciamento do runtime

- ARN da versão do runtime: arn:aws:lambda:us-east-1:runtime:28933846e9cfabf6cf38cf537c134e46a0226f1f0559d98c271ed4b22d8
- Atualizar versão do tempo de execução: Auto

4.5 Gatilho automático (S3 → Lambda)



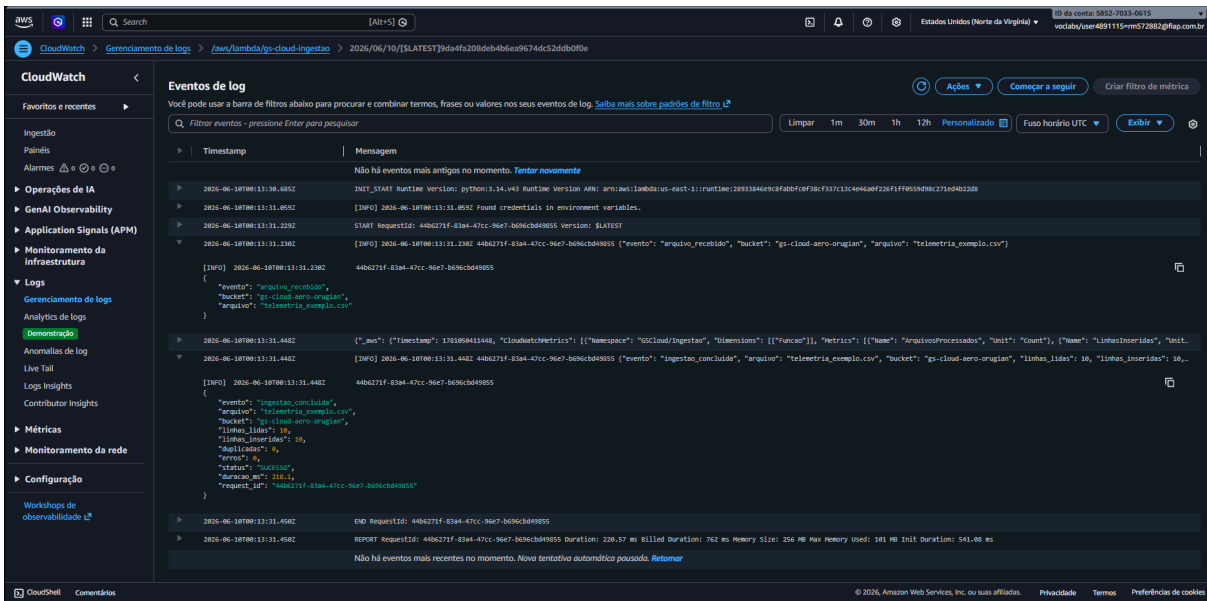
4.6 Ingestão disparada pelo upload



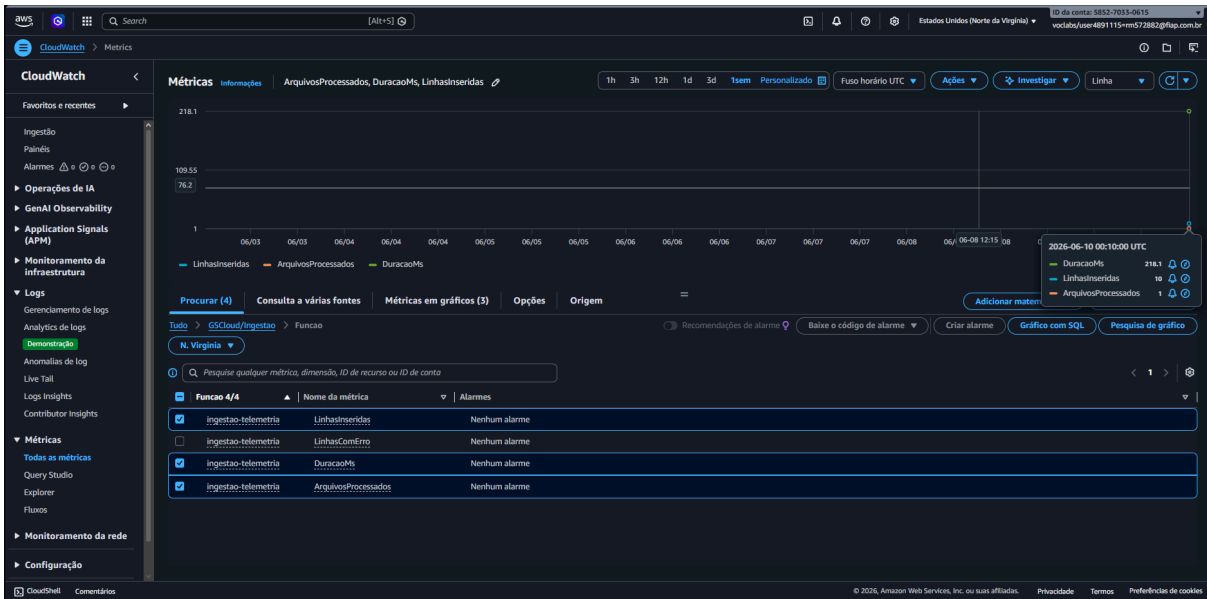
5. Automação evidenciada (CloudWatch)

O upload do arquivo **.csv** no bucket S3 dispara a Lambda **sem nenhuma ação manual**. O CloudWatch registra toda a execução:

5.1 Logs da execução automática



5.2 Métricas da ingestão (namespace GSCloud/Ingestao)

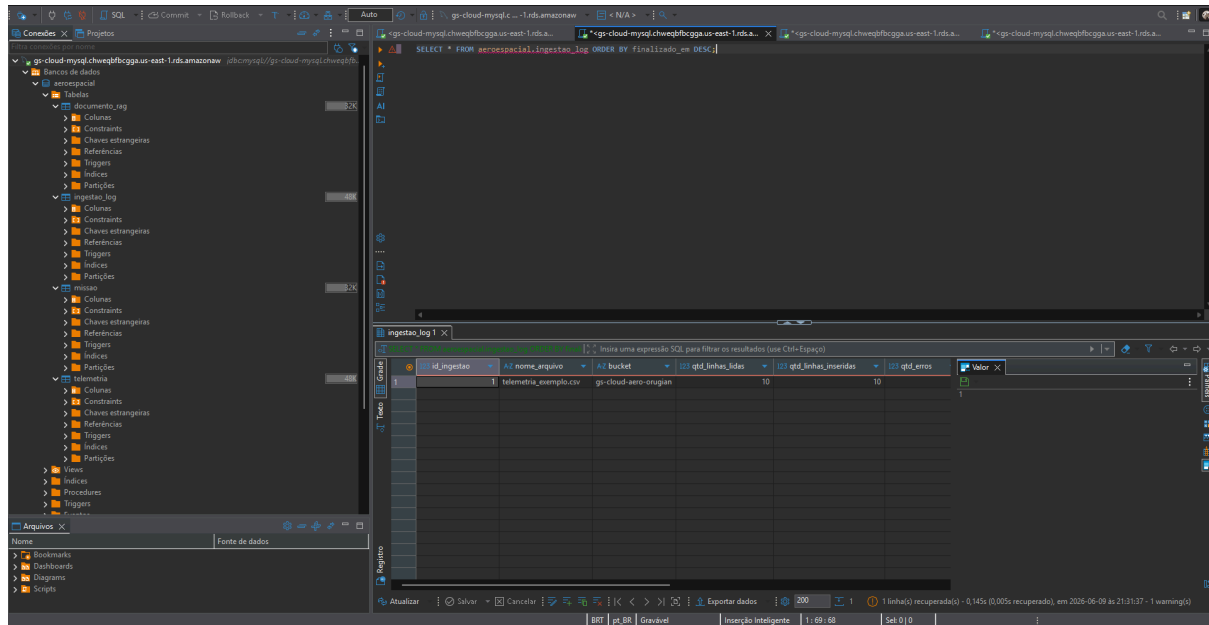


As métricas **LinhasInseridas**, **ArquivosProcessados** e **DuracaoMs** são publicadas via **Embedded Metric Format**: a Lambda escreve JSON no stdout e o CloudWatch extrai as métricas automaticamente — sem chamada de API e sem necessidade de permissão cloudwatch:PutMetricData.

6. Exploração dos dados (DBeaver → RDS)

Após a ingestão, o DBeaver é conectado ao endpoint público do RDS para consultar e explorar os dados inseridos automaticamente.

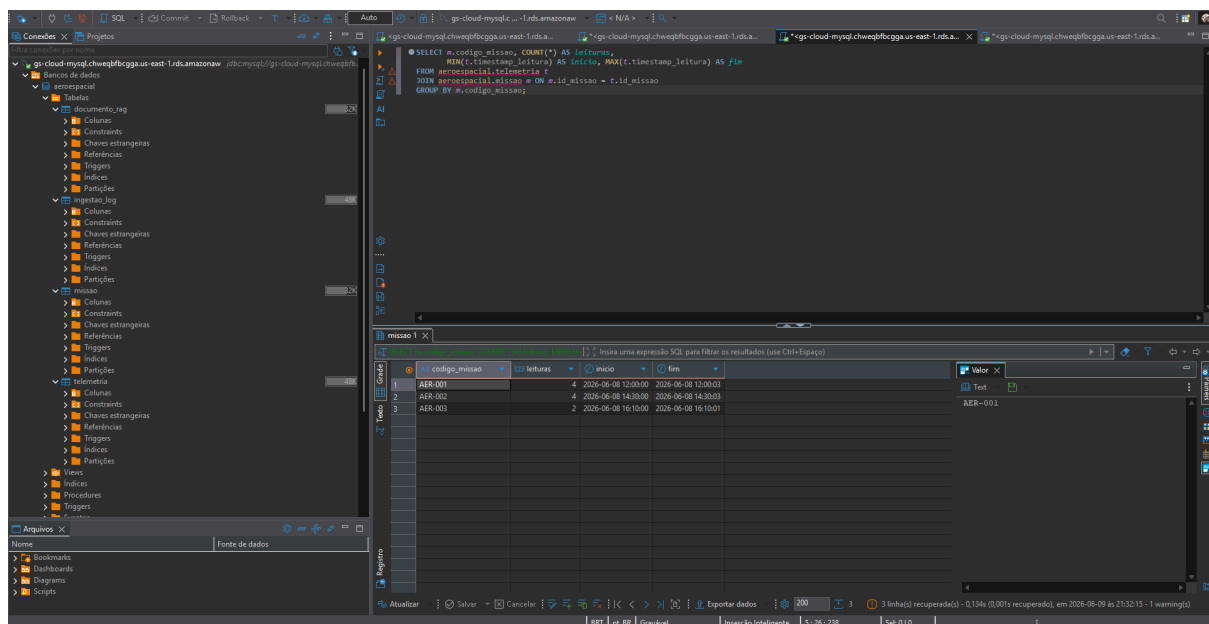
6.1 Trilha de auditoria da ingestão



The screenshot shows the DBeaver interface with the 'ingestao_log' table selected. The table structure is as follows:

id_ingestao	nome_arquivo	bucket	qtde_linhas_idas	qtde_linhas_inseridas	qtde_errs
1	Telemetria_exemplo.csv	gs-cloud-aero-orugam	10	10	1

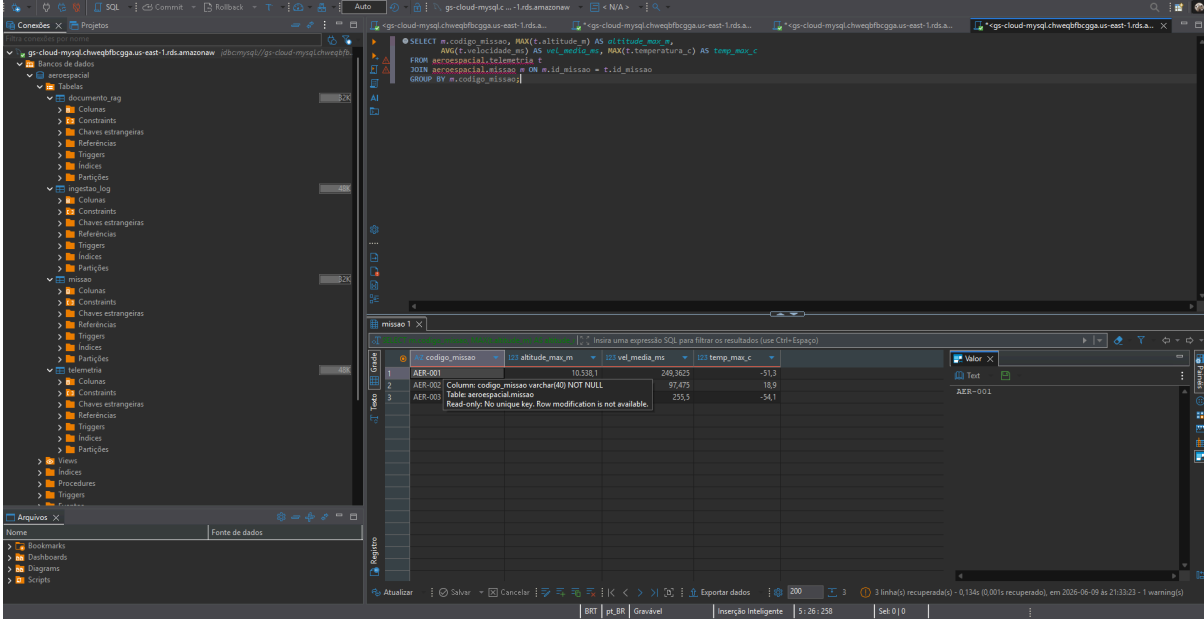
6.2 Volume ingerido por missão



The screenshot shows the DBeaver interface with the 'missao' table selected. The table structure is as follows:

codigo_mission	leituras	inicio	fim
AER-001	4	2020-06-08 12:00:00	2020-06-08 12:00:03
AER-002	4	2020-06-08 14:30:00	2020-06-08 14:30:03
AER-003	2	2020-06-08 16:10:00	2020-06-08 16:10:01

6.3 Análise (picos e médias por missão)



The screenshot shows the AWS Glue console with a SQL query executed on the `missao` table. The query calculates the maximum altitude, average velocity, and maximum temperature for each mission.

```
SELECT m.codigo_missao, MAX(t.altitude_m) AS altitude_max_m,
       AVG(t.velocidade_ms) AS vel_media_ms, MAX(t.temperatura_c) AS temp_max_c
FROM   aereospacial.telemetria t
JOIN   aereospacial.missao m ON m.id_missao = t.id_missao
GROUP BY m.codigo_missao
```

The results table shows the following data:

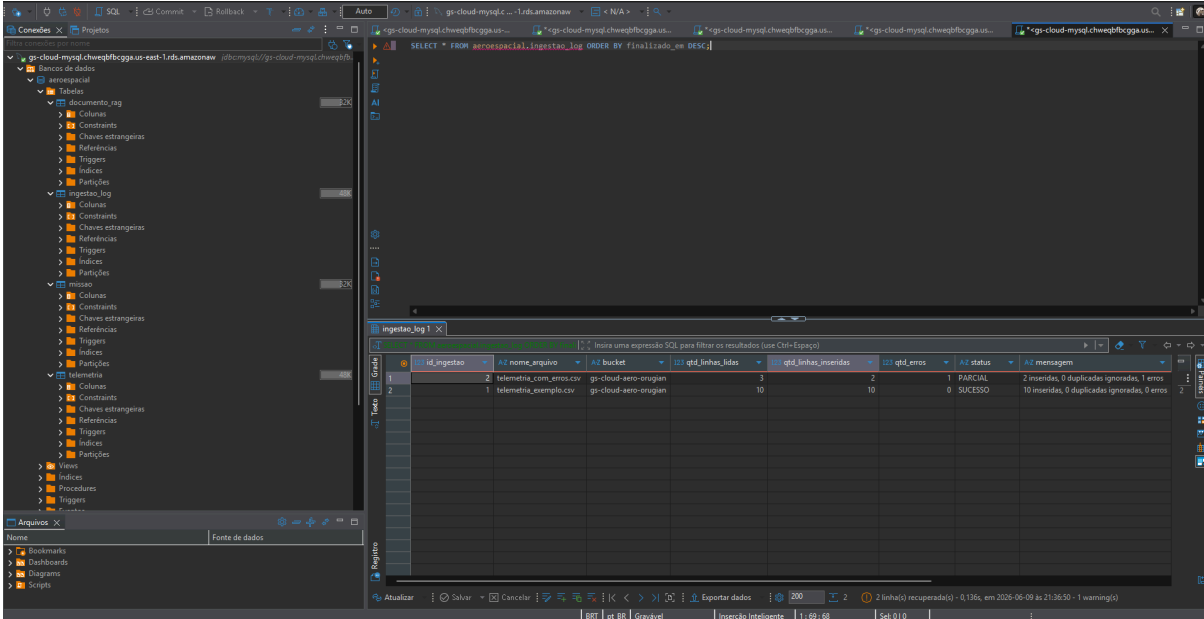
codigo_missao	altitude_max_m	vel_media_ms	temp_max_c
AER-001	10320.1	240.363	-31.9
AER-002	Column 'codigo_missao' varchar(40) NOT NULL	97.0%	18.9
AER-003	Table 'aereospacial.missao'	253.5	-54.1

Warning: Read-only. No unique key. Row modification is not available.

7. Robustez da solução

7.1 Tratamento de erro — status PARCIAL

Quando um arquivo contém linhas com dados inválidos (ex.: **SENSOR_FALHOU** em campo numérico), a Lambda rejeita apenas as linhas problemáticas e prossegue com as demais. O resultado aparece em **ingestao_log** com **status = PARCIAL** e a contagem de erros, sem interromper a ingestão.



The screenshot shows the AWS Glue console with a query on the `ingestao_log` table. The query filters for records where the status is 'PARCIAL'.

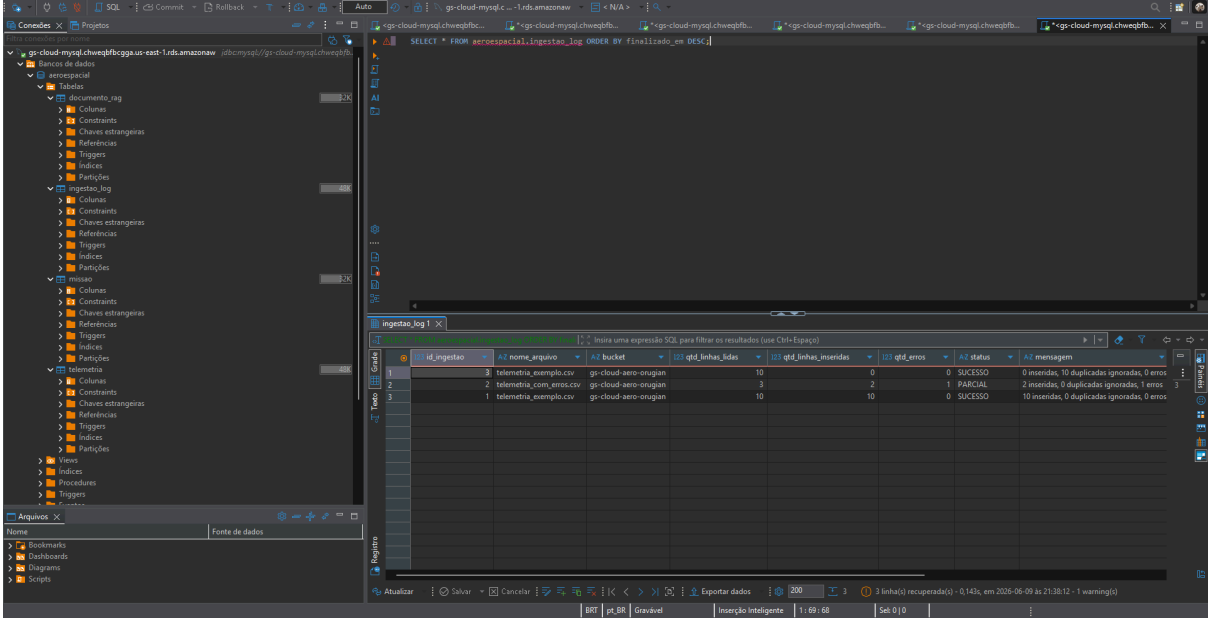
```
SELECT * FROM aereospacial.ingestao_log ORDER BY finalizado_em DESC
```

The results table shows the following data:

id_ingestao	nome_arquivo	bucket	qtd_linhas_lidas	qtd_linhas_inseridas	qtd_errores	status	mensagem
1	telemetria_com_erro.csv	gs-cloud-aero-orugiam	3	2	1	PARCIAL	2 inseridas, 0 duplicadas ignoradas, 1 erro
2	telemetria_exemplo.csv	gs-cloud-aero-orugiam	10	10	0	SUCESSO	10 inseridas, 0 duplicadas ignoradas, 0 erro

7.2 Idempotência — reproprocessamento sem duplicação

O reenvio do mesmo arquivo não gera duplicatas. A chave única (`id_missao`, `timestamp_leitura`) combinada com `INSERT IGNORE` garante que linhas já existentes sejam ignoradas. O `ingestao_log` registra `qtd_linhas_inseridas = 0` no reproprocessamento, provando a idempotência.



The screenshot shows the DBeaver interface with the `ingestao_log` table selected. The table contains the following data:

id_ingestao	nome_arquivo	bucket	qtd_linhas_idas	qtd_linhas_inseridas	qtd_erros	status	mensagem
1	telemetria_exemplo.csv	gs-cloud-aero-orugian	10	0	0	SUCCESS	0 inseridas, 10 duplicadas ignoradas, 0 erros
2	telemetria_corp_eros.csv	gs-cloud-aero-orugian	3	2	1	PARCIAL	2 inseridas, 0 duplicadas ignoradas, 1 erros
3	telemetria_exemplo.csv	gs-cloud-aero-orugian	10	10	0	SUCCESS	10 inseridas, 0 duplicadas ignoradas, 0 erros

8. Conclusão

O projeto demonstrou uma solução completa de ingestão automatizada na nuvem para dados de telemetria aeroespacial. O upload de um arquivo `.csv` no bucket S3 dispara automaticamente a função Lambda, que valida, processa e persiste os dados no RDS MySQL — sem nenhuma intervenção manual. A automação foi comprovada pelos logs estruturados e pelas métricas customizadas do CloudWatch (GSCloud/Ingestao). A exploração dos dados via DBeaver confirmou a integridade e completude das inserções realizadas. A solução ainda demonstrou robustez no tratamento de erros de dados (status `PARCIAL`) e confiabilidade por idempotência (`INSERT IGNORE`). O modelo de dados — com a tabela `documento_rag` contendo campos de `conteudo`, `metadados` e `embedding` — está preparado para a implementação de uma camada de RAG em etapa futura.

9. Preparação para RAG (trabalho futuro)

A modelagem já contempla `missao.descricao` (texto livre sobre a missão) e a tabela `documento_rag` (conteúdo + metadados JSON + embedding JSON), permitindo, numa próxima etapa:

1. Popular `documento_rag.conteudo` com resumos de missão, notas de anomalia e descrições de eventos derivados da telemetria.
2. Gerar **embeddings** via Amazon Bedrock e armazená-los em `documento_rag.embedding`.
3. Expor uma **API serverless** (API Gateway + Lambda) que recebe perguntas em linguagem natural, recupera os trechos mais similares por similaridade vetorial e monta o prompt do LLM — pipeline completo de RAG sobre os dados ingeridos.